AgentMPI: A Message Passing Interface for Multi-Agent Systems

AgentMPI Project

ABSTRACT

Multi-agent LLM systems are in the position parallel computing occupied around 1991: every group writes its own coordination layer, none of them compose, and there is no portable interface one writes a *harness* against. We argue that the interface parallel computing converged on—MPI [34, 35]—transplants, and we specify, implement and evaluate AgentMPI: a protocol for writing multi-agent systems, not a multi-agent system.

The transplant is not mechanical. Five asymmetries between an MPI process and an LLM executor invert several of MPI's answers. The scarce resource is the receiver's *context window* rather than bandwidth, so MPI's eager limit becomes a token-denominated flow-control problem and MPI's "safe program" discipline becomes a mechanically checkable property we call *context safety*. The reduction operator costs seconds rather than nanoseconds, so algorithm selection must minimise operator applications on the critical path rather than message volume: at $p = 128$ the schedule MPI prefers for short messages performs 896 operator applications where reduce-then-broadcast performs 127. And because a shared control plane replaces a point-to-point network, MPI's barrier hierarchy collapses entirely.

We also report a limitation of agent-evaluated reductions that we found rather than predicted, and a mechanism for it. A canonical reduction tree makes an agent reduction *reproducible*; it does not make it *consistent*, because two branches can meet the same conflict and resolve it oppositely with no node in a position to notice. We formalise this as the *locality of merge*, and introduce *conflict lifting*: enlarging the operator's codomain with a conflict set whose join is a semilattice, so the conflicts reaching the root are identical for every tree shape and the root arbitrates each exactly once.

AgentMPI ships as a specification, a reference runtime organised around a six-operation abstract device interface with 3 independent transports, and a conformance suite of 282 protocol assertions and 117 device assertions run unchanged against every transport. We evaluate with real agent populations: paired eight-rank translation runs, a hundred-rank run under executor oversubscription, and deterministic microbenchmarks that fit the cost model. Two results are negative and reported as such.

KEYWORDS

multi-agent systems, message passing, MPI, collective communication, fault tolerance, LLM agents

1 INTRODUCTION

A multi-agent LLM system today is written the way a parallel program was written in 1991. The author picks a coordination pattern—a group chat, a supervisor tree, a directed graph of handoffs—and implements it from scratch against whatever agent SDK is at hand. The result does not compose with anyone else's, it cannot be moved to a different agent host without rewriting, and its failure modes are rediscovered independently by every group that builds one.

Parallel computing left that state by agreeing on an interface [34, 44]. The MPI Forum did not standardise a runtime, a language, or a programming model; it standardised the *calls a program makes* to communicate, and left the process manager, the algorithms and the transport to implementations. That decision is why a program written in 1996 still runs, and why a dozen vendors shipped MPI-1 within two years of the document.

This paper asks whether the same move is available now, and answers that it mostly is. AgentMPI is a protocol you write a harness *with*, in the sense that MPI is a library you write a parallel program with. It does not decide how many agents to run, what they should do, how to prompt them, or how to recover; it provides the mechanisms with which those decisions are expressed. It is deliberately semantics-thin—a message is an opaque body plus an envelope—which is a considered rejection of the KQML/FIPA-ACL lineage whose mentalistic semantics proved unverifiable, in favour of MPI's stance: standardise the mechanism, leave meaning to the application.

The transplant is not mechanical. Five properties of an LLM executor differ from an MPI process (§3), and each forces a deviation. Two of MPI's algorithm-selection rules invert. One of MPI's cost terms, the reduction operator, goes from free to dominant. One of MPI's non-goals, fault tolerance, becomes the common case. And one resource with no MPI analogue—the receiver's context window—turns algorithm selection from an optimisation into a feasibility test.

Contributions.

(1) A specification (§4) that maps MPI's chapters onto agent coordination and states, for each, whether the mechanism transfers, transfers with a changed unit, or does not transfer. It is written in the standard's own idiom: normative text with explicit rationale.
(2) *The locality of merge* (§5), a limitation of agent-evaluated reductions that we observed in a real run and could not previously name, together with two mechanisms—conflict lifting and invariant verification—and a shape-independence property for the first.
(3) A cost model and re-derived selection rules (§6) in which the operator term dominates, with the resulting inversions stated and measured.

(4) A reference implementation organised around a narrow waist (§7): six device operations, 3 independent transports, and one conformance suite run against all of them. The suite is the artifact that makes this a specification rather than a library, and it found defects that only exist on one transport.
(5) An evaluation with real agent populations (§8), including two negative results we report because they are the informative ones.

2 BACKGROUND: WHAT MPI STANDARDISED

MPI was assembled in 1992–1994 [14, 34, 44] from a field of incompatible message-passing libraries—p4, PARMACS, Chameleon [24], Zipcode, PVM, Express, NX—each of which solved part of the problem and none of which was portable. The Forum's rules were unusually restrictive and unusually effective: standardise existing practice rather than invent; decline everything not yet understood.

MPI-1's *non-goals* were as consequential as its goals. It is not a language. It does not provide shared memory, debugging, I/O, threads, task management, or fault tolerance. Declining those kept it small enough to implement.

Three of its decisions matter here.

Communicators. A communicator pairs a group with a private *context* that partitions the message namespace. Pre-MPI libraries addressed messages by destination and tag, so a library's messages could be intercepted by its caller whenever they collided on an integer. MPI's designers retrospectively identify the context as the reason MPI could support libraries at all [23, 27, 41]. The agent-world version of that collision is not hypothetical: a reviewer's critique intended for the author of module A being consumed by the author of module B is the same bug, and the "one shared group chat" architecture makes it certain rather than merely possible.

Safe programs. MPI deliberately refuses to guarantee buffering. A program that completes only because an implementation happened to buffer is called *unsafe*, and the standard's advice is to test a program by replacing every standard-mode send with a synchronous one. The test is mechanical and almost nobody runs it, because the penalty is a deadlock on someone else's machine.

Performance portability. The standard specifies semantics; implementations choose algorithms. The Hockney model $T = \alpha + \beta n$ and its successors [3, 13] let an implementation pick a collective algorithm by message size and process count [38, 42]. Evidence on which parts of MPI applications actually use [32] is what motivates our conformance levels.

3 WHY AN AGENT IS NOT A PROCESS

Table 1 lists them. Their consequences run through the rest of the paper: A2 forces the eager threshold to be denominated in tokens (§4.2); A3 inverts algorithm selection (§6); A4 motivates quorum collectives; A5 requires the whole fault-tolerance chapter; A1 requires reduction *reproducibility* and, separately, *consistency* to be declared properties (§5).

Table 1: The five asymmetries that drive every deviation.

	MPI process	AgentMPI executor
A1 Determinism	deterministic	may produce a different, plausible, wrong result
A2 Scarce resource	bandwidth	**context window**
A3 Operator cost	nanoseconds; free	seconds to minutes; dominant
A4 Latency	tight	heavy-tailed; max of p samples
A5 Failure	fail-stop, rare, fatal	frequent, partial, sometimes silent

One asymmetry runs the other way, and is easy to miss. In MPI the control plane is a point-to-point network. In AgentMPI it can be, and in our implementation is, a *shared medium* every rank can read. That single structural difference collapses MPI's barrier hierarchy: a counting barrier costs $2(p-1)$ operations against dissemination's $p\lceil\log_2 p\rceil$—510 against 2,048 at $p = 256$—so it is cheaper at every size, and it is also the only algorithm that can name the ranks that did not arrive. MPI avoids counting barriers at scale for a reason that does not apply here.

4 DESIGN

We summarise the specification; the normative document accompanies the artifact.

4.1 Ranks, epochs, and identity

A rank is a *durable role*, not a process: a number, a mailbox, a context budget and an accumulated ledger. Its physical embodiment is an *executor* bound to it for a bounded interval, identified by an *epoch*. In MPI a rank *is* a process and the identification is harmless because MPI processes live for the whole job; an agent session does not, so tying rank identity to session identity would make every timeout a communicator rebuild.

The epoch is a fencing token. Leases alone are insufficient: an executor cannot know its lease expired mid-step, so between expiry and its next call two executors believe they own the rank. A monotone token checked on every operation closes that window and makes a zombie harmless rather than corrupting.

Identity is *ambient* but must be *assertable*. This is a correction we were forced into. Ambient identity eliminated the error we designed it to eliminate—agents passing the wrong rank—and replaced it with a worse one: agents silently *being* the wrong rank, because on one host shell sessions were shared between concurrent agents and the rank variable was overwritten between calls. The fix is not to abandon ambient identity but to make the assertion cheap.

4.2 Flow control in tokens

Every rank has a context budget; delivering a body charges it. Below an *eager threshold* (700 tokens by default) a body is pushed into the receiver; above it, the receiver gets an envelope and a content-addressed handle and materialises explicitly. This is MPI's eager limit with the unit changed from bytes to tokens, and it exists for the same reason: below the limit, pushing unsolicited is cheaper than a handshake; above it, the receiver's buffer—here, its attention—is too precious to fill without permission.

Long contexts are not free even when they fit: model accuracy degrades with position and length [33], and serving systems treat the window as a managed resource [29]. Both are reasons to account for it in the protocol rather than in the prompt.

Two things do not transfer. First, in MPI eager and rendezvous are invisible because both deliver the same bytes; here they differ in *what arrives*, so they must be visible. Second, exceeding the buffer does not deadlock, it silently degrades the receiver's output.

That second point makes MPI's safe-program discipline more important here, not less, so we mechanise it. Define a program *context-safe* if it completes for every assignment of unexpected-message budgets, however small. The reference implementation runs a harness's communication skeleton under zero-buffer semantics, reports the ranks that wedge and any cycle among them, and re-runs with every send declared rendezvous so it can say whether the harness is repairable by transport choice alone. A ring in which every rank sends before it receives—the shape agent harnesses write by default—is diagnosed with its cycle named and the repair stated.

4.3 Views and contracts

An MPI derived datatype is a declarative description of how to access a buffer, and its value is that the access pattern is data the library can optimise rather than control flow it cannot see. The scarce resource here is context, so the analogous question is not "which bytes" but "which tokens, and how few". A *view* is a deterministic, model-free projection: `head:n`, `grep:pat`, `keys:a,b`, `outline`, `stat`. Semantic compression is deliberately *not* a view, because it costs an operator application the harness must be able to see.

Contracts transplant the second job of MPI datatypes: a type signature checked at both ends, turning silent corruption into a loud error. A contract may also require a payload to identify itself, which turns a misrouted scatter slice into an error at the receiver rather than a plausible wrong answer three phases later.

4.4 Collectives

Labelled, not positional. MPI identifies the kth collective by program order, which is safe when the caller is a compiled program. An executor's program order is not: it retries, skips, and reorders. Positional identification would pair one rank's second call with everyone else's first and return a confidently wrong result. Labels cannot make a reordered schedule complete—nothing can—but they make the mismatch *named*.

Gather returns a *manifest*, not a concatenation. At $p = 128$ with 4,000-token contributions, an inlining allgather charges one rank 508,000 tokens and moves 65,024,000 in total; a handle-based one charges 5,080 and moves 650,240, a factor of 100 in peak residency. Concatenation is not a reasonable default at any interesting p.

Barriers carry a *policy*—wait, raise, proceed, shrink, revoke—because only the harness author knows whether a missing participant degrades the result or kills it. Collectives may carry a *quorum*, which *releases* without *closing*: a straggler still passes through, because a quorum that closed would guarantee that precisely the slowest ranks fail.

4.5 Windows

Every multi-agent system grows a shared scratchpad, and every one gets the concurrency wrong the same way: two agents read it, both edit a private copy, and the second write discards the first's work. MPI-3's one-sided chapter supplies both the vocabulary and the tools. `put` is the blind overwrite that loses work; `accumulate` combines atomically, so "union this finding into the shared findings" needs no lock; `compare_and_swap` is how work is claimed, and unlike a lock it cannot be held by a dead executor.

Two departures. Locks are *leased* and carry a fencing token, because an executor can wander off where an MPI process cannot. And enumeration is free while reading is not: listing a window's keys with their sizes and writers costs a few tokens per key, which is what makes a blackboard usable by an executor with a bounded context.

4.6 Fault tolerance

ULFM's stance [5, 6] adopted wholesale: expose failure, provide mechanisms not policy. `revoke` is the non-obvious primitive—when a rank fails the *survivors* are the problem, blocked inside collectives that can never complete—and revocation makes them all fail fast so they reach the recovery path together. `shrink` derives a communicator over an *agreed* survivor set, and also offers FT-MPI's [16] in-place mode, because renumbering invalidates a rank-to-work mapping that for agents is baked into prompts.

Recovery is by *briefing*, not checkpoint. There is no memory image to restore, so process checkpointing has no analogue; what does is durable-execution replay. The dominant failure here is a confident wrong answer, which is the exact analogue of silent data corruption, and HPC's answer to that is not checkpoint/restart—which faithfully preserves the corruption—but algorithm-based fault tolerance [8, 28]. A replacement is given the answers to five questions it cannot guess: what was I assigned, what did I publish, what did I receive, what did I promise that is outstanding, and what did I record for myself.

Detection is lazy, local, and two-phase; it is an eventually perfect detector in Chandra and Toueg's classification [12], and the leases carry fencing tokens for the reason Gray and Cheriton give [20]. A single-phase lease detector convicted 1091 times in twenty minutes on a real host, because a thinking executor and a dead one look identical. A blocked rank must renew its own lease while waiting; omitting that once made a barrier convict everyone who arrived early.

5 THE LOCALITY OF MERGE

MPI requires a user reduction operator to be associative and then reorders freely. The guarantee is deliberately weak—floating-point `MPI_SUM` is not bitwise reproducible across process counts [2]—and practitioners accept it because the discrepancy is bounded by rounding error.

An operator implemented by a language model is not associative, is lossy, is expensive, and is non-deterministic, and the discrepancy is bounded by nothing. AgentMPI therefore makes the algebra *declared* rather than assumed and refuses schedules the declaration does not license. That much is straightforward.

What is not straightforward is a failure we found rather than predicted.

> In an eight-rank reduction over interface proposals, two branches of the tree independently encountered the *same* conflict—which module defines the shared exception type—and resolved it in *opposite* directions. Both resolutions were recorded, so the merged result contained two contradictory rulings under one identifier, and the tree had no way to notice: each merge saw a locally consistent pair. Two ranks then implemented different halves of the result. Separately, the same reduction dropped four of eight modules from one section of its output despite an explicit instruction never to drop a module.

A canonical tree shape makes a reduction *reproducible.* It does not make it *consistent*, and conflating the two is the mistake.

The observation, stated generally. Let a reduction be a fold of a binary operator $\oplus$ over a tree T whose leaves are the ranks' contributions, and call a predicate φ a *global invariant* if its truth depends on the whole leaf multiset. An internal node of T sees exactly two operands. If two nodes u and v, neither an ancestor of the other, both encounter evidence bearing on φ, then no node of T is in a position to reconcile them: their lowest common ancestor receives u's and v's *outputs*, and the information that would reveal the inconsistency—what each decided, and why—is not in $\oplus$'s codomain. Enlarging the tree cannot help. Only enlarging the codomain can.

Conflict lifting. Let the operator's codomain be $V \times C$, where C is a set of undecided conflicts, and require that an operator which cannot decide a contested item from its two operands alone *lift* it into C rather than decide it. Let C's join be set union.

Proposition 5.1 (Shape independence). *The conflict set arriving at the root of a fold over $\oplus$ is the same for every tree shape over the same leaf multiset, and no contested item is decided more than once.*

The argument is two lines. The conflict component's join is associative, commutative and idempotent, so it is a semilattice fold, and a semilattice fold is independent of the order and shape of the fold. The value component never decides a contested item, so it cannot decide one twice. Arbitration at the root therefore decides each conflict exactly once, over the same set, whatever the schedule.

We verify this exhaustively: over every binary fold order at $p \leq 6$ (all Catalan$(p-1)$ shapes) and over random orders to $p = 32$, the conflict set is invariant, while a control operator that decides locally is not.

Why it is cheap. Lifting buys the "one ruling per key" invariant for one extra operator application and $O(|C|)$ tokens. The alternative that also buys it—a serial chain, in which the accumulator carries every prior decision—costs $p-1$ critical-path applications against a tree's $\lceil \log_2 p \rceil$: at $p = 64$, 63 against 6.

Implementation warning. Represent the lifted state as a map from key to the *set* of distinct values seen, and derive the value/conflict presentation from it. The obvious implementation—lift on first disagreement and remove the key from the value—is *not* idempotent: a third contributor finds the key absent and reinstates it, so the conflict set depends on fold order after all. We shipped that version, and its own shape-independence test caught it.

The other half. "Never drop a module" is not a conflict any pair of operands exhibits—every local merge is individually faithful—so lifting is blind to it. It is a property of the leaf multiset and the result, so an operator may declare a *global invariant* that is checked once, after the reduction closes, and reported with the missing items named.

Neither mechanism makes an agent operator correct. They make two specific, observed, expensive failure modes into loud errors, which is the most a protocol can honestly claim.

6 COST MODEL AND ALGORITHM SELECTION

For an operation moving n tokens,

$$T = \alpha + \beta n + \gamma k + \lambda, \qquad C = c_{\mathrm{tok}} n + c_{\mathrm{op}} K,$$

where γ is the cost of one operator application by an executor, k the number on the critical path, K the number in total, and λ the executor's own heavy-tailed think time.

Measured on the reference implementation (§8.1), $\alpha =$ 0.730 ms and $\beta = 0.480\,\mu$s/token on the SQLite transport,

Table 2: Total operator applications, allreduce. Recursive doubling and reduce-then-broadcast put the same number on the critical path.

p	8	16	32	64	128
recursive doubling	24	64	160	384	896
reduce + broadcast	7	15	31	63	127
ratio	3.4	4.3	5.2	6.1	7.1

giving a half-bandwidth point of 1,521 tokens—above the size of a typical agent artifact, so latency-optimal algorithms win over a wider range of sizes than in MPI. Against them, γ for an LLM merge is seconds to minutes. At $p = 128$ the entire protocol cost of an allreduce is 15.09 ms, which is 0.215 % of the total against a one-second operator and 0.0072 % against a thirty-second one.

Two of MPI's rules invert.

(a) Runtime operators want no tree. When the implementation can apply the operator—set union, JSON merge, max—a shared control plane folds every contribution in place: one round, zero messages. A tree adds rounds and buys nothing. This is the in-network-aggregation regime, and it is the common case for the operators harnesses *should* be using. Sweeping γ confirms it: at $\gamma = 0$ the winner is `flat`, and at $\gamma = 30$ s it is `reduce_bcast`, with the flat schedule 10 × behind (1890.0 s against 180.0 s).

(b) Agent operators want MPI's tree, and reject MPI's best tree. A binomial tree puts $\lceil \log_2 p \rceil$ applications on the critical path against a chain's $p-1$; the catalogue also includes Bruck's algorithm [9] and Rabenseifner's [39], and the scan schedules follow Ladner and Fischer [31] and Blelloch [7]. But recursive-doubling allreduce—MPI's standard choice for short messages [38, 42], precisely because redundant arithmetic is free—performs $p\lceil \log_2 p \rceil$ applications *in total* (Table 2) for the same critical path. The algorithm that wins for bytes loses badly for agents, and it loses on the *price* axis, which is why the model reports time and price separately.

Admissibility precedes optimisation. An algorithm whose peak per-rank residency exceeds a context budget is *infeasible*, not slow. At $p = 128$ with 4,000-token payloads, every inlining allgather in the catalogue is rejected with its peak residency named; the handle-based variants are admissible. MPI has no analogue: there, every algorithm runs.

7 IMPLEMENTATION

7.1 The narrow waist

MPICH's portability comes from an abstract device interface separating portable MPI semantics from transport [10, 21, 22]; Open MPI makes the same move with MCA frameworks. AgentMPI copies the structure, for a different reason: an interface with one implementation is a library, and what would make this a standard is that the semantics can be stated and *checked* independently of any transport.

Six operations sit at the waist: `append` (durable, totally ordered), `match` (atomic first-fit claim), `scan`, `cas`, `lease`, `clock`. Everything above—communicators, matching, collectives, windows, the ledger, revoke/shrink/agree—is written in terms of those six. Why not fewer: `append` and `scan` suffice for a single writer, but a receive must not be handed to two ranks, and that atomicity cannot be recovered above the device. Why not more: window history is a `scan` over versions, the failure detector is a `scan` over leases, and the deadlock detector is a `scan` over blocked receives.

Three transports ship, sharing no code below the interface: SQLite in WAL mode, a filesystem journal using one file per record with an advisory lock, and an in-process dictionary. 10,865 lines of implementation sit above the waist.

7.2 The conformance suite

117 device assertions and 282 protocol assertions, each naming the specification section it enforces, run unchanged against all 3 transports. The suite uses only public interfaces, so pointing it at a different implementation means changing one fixture.

It earns its keep. In its first pass it found six defects, of which two matter beyond this codebase. An administrative kill was retractable by its victim's next heartbeat—which would have made every fault-injection measurement meaningless. And SQLite's type affinity was returning integers as strings for four indexed fields, so a wildcard receive posted as -1 read back as `"-1"` and silently stopped matching: a divergence that existed on *one* transport and would otherwise have surfaced as an inexplicable protocol bug months later. That is precisely what a suite parametrised over every transport is for.

7.3 The command binding

For an LLM executor the binding is a command-line tool, because that is the only interface an agent reliably has to a stateful library: it cannot hold a handle across turns, cannot link a shared object, and its "function calls" are invocations whose output lands in its context window. Eight properties are normative, each because its absence cost a run: assertable identity; an identity echo on every call; errors that carry a concrete next action and a retryable flag; internal retry; sizes in tokens; a free path to disk on every operation that hands back a payload; the manual as a command; and *print only commands that exist.* That last one has a test that walks every emitted command string through the real parser, because an early reduction directive printed a subcommand spelled with a space where the real one is hyphenated, and roughly ten agents reported it, several while peers were blocked behind them.

8 EVALUATION

8.1 Microbenchmarks

No model is involved, deliberately: the job is to measure the *protocol* so the agent experiments can attribute their time correctly.

Fitting $T = \alpha + \beta n$ by ping-pong regression over payloads from 1 to 12,800 tokens gives $\alpha = 0.730$ ms on SQLite, 3.947 ms on the filesystem journal, and 0.096 ms in memory. The half-bandwidth point is 1,521 tokens on SQLite and 8,104 on the journal.

One result was not anticipated: β agrees to within 1.5 % across all three transports. The per-token cost is serialisation and token counting *above* the waist, not transport below it. Only α is a property of the device—which is a useful thing for the cost model to know, and an argument that the waist is in the right place.

8.2 E1: translating a book

The task is native translation of a book, chosen because it is *almost* embarrassingly parallel. Each passage translates independently; what does not is the rendering of names and coined terms recurring across passages. Ignore the coupling and the Tin Woodman acquires four names; serialise to avoid it and you pay p executor latencies.

We split *The Wonderful Wizard of Oz* into 100 passages and extract 21 recurring terms by capitalisation frequency, filtered by a model-free rule (a candidate that also appears lower-cased anywhere in the corpus is dropped). The harness has six phases, each one collective: scatter the passages; an agent phase proposing renderings; an allreduce with `union`, which lifts disagreements rather than deciding them; arbitration at the root; a broadcast of the binding glossary; an agent phase translating; a gather.

The protocol is in the harness, not the prompt. An executor's entire obligation is to read a prompt file, write a result file, and submit; the worker bootstrap prompt mentions no communicator, collective, barrier or glossary. That is the design claim stated as an artifact, and it is why the same experiment runs against a deterministic function or a hundred live agents without changing a line.

Scoring is mechanical. For each recurring term, we find which rendering each passage's translation used and take the modal rendering's share, weighted by how many passages contain the term. The candidate renderings searched for are pooled across *all* arms compared, which matters: our first scorer built the vocabulary from each run's own term sheets, and since only the treatment arm produces term sheets, the control was scored against an almost-empty vocabulary and flattered by it.

The mechanism worked, and scaled as predicted. At $p = 8$ the reduction agreed 17 terms unanimously and lifted 3 as conflicts; at $p = 32$ it agreed 8 and lifted 12. Quadrupling the population quadrupled the disagreement it found, which is what one would expect and what the conflict machinery exists to surface. The root arbitrated each conflict once, at both sizes.

Every one of the 12 conflicts was the same kind: whether the Spanish definite article belongs in the rendering—*el Espantapájaros* against *Espantapájaros*, *la Ciudad Esmeralda* against *Ciudad Esmeralda*. Not one was a disagreement about the noun.

Table 3: E1, two paired arms at two sizes. Both metrics are null at both sizes, and the collective costs wall clock at both.

	$p = 8$		$p = 32$	
	gloss.	ctrl	gloss.	ctrl
Consistency	1.000	1.000	0.994	0.994
strict	0.9474	0.9474	0.9509	0.9632
Terms scored	10	10	15	15
Wall clock (s)	400	109	346	255
Result tokens	6,648	6,082	26,480	24,406

The result is null, at both sizes. Table 3. Weighted terminology consistency is 1.000 in all four arms; under the strict metric that counts the article as part of the decision, the control arm at $p = 32$ scores marginally *higher* than the treatment (0.9632 against 0.9509). The glossary cost 3.7 × the control's wall clock at $p = 8$ and 1.4 × at $p = 32$.

We had predicted that the null at $p = 8$ was a scale effect and that a larger population would surface terms with weaker priors. The prediction was half right and instructive in the half that was wrong. The population's *stated* conventions did diverge more at scale, exactly as predicted—four times as many lifted conflicts. But that divergence did not reach the artifact, because the thing the translators disagreed about was the definite article, and in running prose the article is determined by the grammar of the sentence rather than by a lexical choice. Each translator wrote *el Espantapájaros* or *Espantapájaros* as the sentence required, whichever they had declared.

So the collective correctly identified a real disagreement in what the population said it would do, and that disagreement turned out not to matter for what the population produced. This is the mirror image of the caution in §5: agreement is not correctness, and here disagreement was not incorrectness either. The operational lesson for a harness author is the one we did not expect to be reporting—before paying a collective to remove a coupling, measure whether the coupling is costing anything.

One threat runs the other way and is worth stating for that reason. An executor in the treatment arm reported, unprompted, that it capitalised compass words mid-sentence because the glossary lists them capitalised and it reasoned the check would be mechanical; it chose a less natural rendering to match what it believed the scorer looked for. That is Goodhart's law inside the experiment, biasing the treatment arm upward on the metric we report (79% against the control's 70% on mid-sentence capitalisation). The treatment did not beat the control even so, which makes the null a floor rather than a point estimate. The general lesson is one for harness authors: a metric an executor can see is a metric an executor will aim at.

We note the other threats. This is $n = 1$ per cell, one model family, one language pair, and one corpus whose recurring terms are famous enough that a strong model has seen them.

Table 4: E2: each row is a claim from the specification, measured.

Claim	Measured
A rank killed before a collective must not hang it	3 killed, 9 contributors, omission recorded
Revocation frees a survivor blocked *inside* a collective	freed after 1.32 s with `ERR_REVOKED`
Concurrent shrinks converge	11 ranks shrinking, 1 communicator
A replacement can resume from the briefing alone	5/5 published cells, plus the open collective and the memo
Claimed work is never done twice	24 tasks, 23 done, 0 duplicated

A task whose terminology is genuinely novel—an internal codebase, a new API—is where we would expect the collective to pay, and we have not run it.

8.3 E2: fault tolerance

The question here is not how a model behaves under failure—E1 and the conformance suite cover that—but whether the recovery machinery does what §4 says when ranks die at the worst moments. That has a definite answer, and getting it means killing ranks at controlled points many times, which is exactly what one should not pay a language model to sit through. Scripted ranks, $p = 12$, five scenarios, each a specification claim stated as a measurement. All 5 of 5 hold.

The last row is the one worth reading carefully (Table 4). With one rank dying mid-claim, 0 tasks were done twice and 1 was left orphaned: claimed by a dead executor and never finished. That is exactly what the protocol promises and no more. Compare-and-swap guarantees a task is not done twice; finishing a dead claimant's task requires a supervisor, and reporting the orphan count is how a harness author learns how much they must still do themselves.

8.4 Oversubscription, and a launcher constraint

Our agent host caps concurrent sessions at ten. This is not an accident of our setup; every host we have used imposes some such cap, and it is exactly the condition that leaves a job with ranks nobody will ever start. The $p = 32$ runs above are served by 8 executors at oversubscription four.

The protocol's answer is oversubscription, and it has a precise MPI analogue: `mpirun -np 100` on eight cores runs a hundred ranks on eight cores. It works here for the same reason it works there—a rank is a durable role whose state lives outside its executor—and it is only available because the protocol is in the harness. An executor serving ten ranks *agent-side* would have to be inside ten barriers at once and would deadlock immediately; here the harness thread blocks in the collective and the executor blocks on nothing.

The specification's other answer is the *join deadline*: a rank's lease starts when the rank is *requested*, not when its executor first calls in. Without it, a rank whose executor never starts has no lease to expire, so it is neither alive nor failed and every peer waits for it forever. With it, a launcher that can start six of twenty-two ranks produces sixteen detectable no-shows.

8.5 Two identities, and the one we thought was checked

Two identities travel with every task in these experiments. The *protocol identity* is the rank an operation acts as, which the binding lets a caller assert with `--expect-rank`. The *provenance label* is an environment variable we added so that a scale claim would have per-executor evidence; it is outside the protocol and nothing checks it.

On a host where shell sessions are shared between concurrently running agents, both drifted. Across 199 tasks and 19 executors the provenance label was wrong on 18 of them (9.0 %). In every run with one executor per rank, and in the $p = 32$ runs at oversubscription four, it drifted 0 times.

We first reported that the protocol identity held because it was asserted. That was wrong, and two executors told us so. One wrote: "the `next` call carried `--expect-rank 3` and still returned rank 2's identity." Another diagnosed it exactly: "the guard didn't fire because the clobber replaced `--rank` and `--expect-rank` *together*." Both are right, for two independent reasons, and the reasons are more useful than the claim we had made.

The implementation defect. The assertion was wired into the library API and into the ordinary CLI operations, and *not* into the `worker` subcommand — the only surface any executor in these experiments ever touched. The conformance suite covered the paths we had thought about. What actually caught the one misroute that was caught was a different and weaker mechanism: the broker's check that a submitted task belongs to a rank the caller serves. That check is a serve-set membership test, not an identity assertion, and it fires only at submit, after the work is done.

The design defect, which is the interesting one. An assertion compares two values the caller supplies. A corruption that rewrites *both* the acting rank and the assertion leaves them consistent, so the check passes and the operation acts as somebody else. No amount of asserting fixes this, because the assertion is made of the same material as the thing it asserts about.

The mechanism that does fix it was already in the specification and already implemented: a per-rank *launch token*, issued by the launcher and recorded in the job, which cannot be made self-consistent by an accident of the environment. We had the mechanism and not the property, because the path that needed it did not call it.

What the executors did instead. Both misrouted claims were handled correctly, by agent judgement rather than by the protocol. One executor abandoned its task with an accurate reason. The other declined even to abandon it, reasoning that `give-up` would also have acted as the wrong rank and would have discarded another executor's work, and let the claim lease-expire instead — which the broker requeued, and its rightful server completed. That is a better decision than the protocol was in a position to enforce, and we would rather not have to rely on it.

All four defects are fixed and regression-tested: identity is asserted on the worker path, the emitted `submit` and `give-up` commands carry `--expect-rank` rather than leaving it to the agent to remember, and the identity flags are accepted on either side of the subcommand. The last of those is its own lesson (§9).

8.6 E7: the production translation, from one machine to eight

E3 wrote the protocol for this task and could not staff it. Its executors were agent-host sessions; the host capped concurrent sessions at ten; and every run above $p = 16$ spent its wall time waiting for an executor to exist (the record is on the branch that wrote it). E7 keeps E3's design—the same collectives, for the same reasons—and changes what a rank *is.* A rank is an operating-system process. Its executor is a chat-completions call with a bounded tool loop. It is started by `ampirun`, the process manager the standard leaves unspecified, on one machine or across several. Executor supply stops being the experiment's limiting resource, so for the first time the population can be scaled against a fixed workload: the same 2013 Russian biography, 3 scales from $p = 16$ on one machine to $p = 64$ over eight.

What changed, and why. Four things distinguish E7 from E3, and each is a protocol mechanism rather than a prompt.

The unit is the paragraph. A page-cut book has 95 prose pages; a paragraph-cut one has 1232, so p can reach 256 with two to five paragraphs a rank. Segments stay contiguous and character-balanced, and the seam exchange keeps its meaning.

Arbitration is agent-evaluated and parallel. E3's root settled lifted conflicts with the runtime's default rule, which chooses a candidate and exercises no judgement. Here the root asks a model—but not alone. After the `allreduce` every rank holds the same conflict set, so rank r arbitrates batch r of it, the rulings are gathered to the root, and the root commits them with `op_arbitrate` exactly once. The serial step that would otherwise grow with the population is spread over it; what remains serial is one `gather` and one commit.

Results are checkpointed and collectives replay. Every agent result is written to a window before the rank proceeds. A rank whose process dies is restarted by `ampirun` through the runtime's `respawn`, receives a new epoch, and re-enters the same program: each collective it already joined returns its stored result (the completion is traced `replayed`, with its wait measured from re-entry), each task it already finished is read back, and it resumes at the first thing its predecessor did not finish. With `--die-fraction` a chosen fraction of ranks exits mid-phase, so the cost of the lifecycle failure every multi-agent postmortem reports becomes a measured quantity.

Shared state under a lock. A translator who settles a term the glossary did not cover records it in a per-chapter ledger under an exclusive, leased, fenced window lock, first writer wins. The merge needs a judgement `accumulate` cannot make—a term already present with a different rendering is a clash to record, not a value to overwrite—so this is the one place the harness locks, and the lock's contention is in the trace.

The executor. A raw endpoint has no body. The executor (`ampi/model.py`) gives it the smallest one that suffices: an encyclopaedia search, an article, a page fetch, called through the provider's function-calling schema in a loop bounded both by steps and by cumulative prompt tokens. Contract violations are repaired in the same conversation rather than by a fresh attempt. Every call's exact prompt and completion tokens, tool calls and cost land in the trace as `task.*` events, which the analysis reads as it reads the broker's claim/submit pair. Two things the executor taught us are recorded as defects fixed. A tool loop whose tools are silently withdrawn produces empty replies; withdrawing them in the model's own turn does not. And a rate limit is not a failure of a task: the first production attempt lost six ranks to a twenty-requests-a-minute cap because 429s were retried like transport faults, six times and then never. A rank waiting on a rate limit is a rank blocked, exactly like one blocked in a collective, and the wait is now bounded by patience rather than by a count.

The population is heterogeneous by necessity. The provider caps every capable model at twenty requests a minute *per model* for the account these runs used. A population of sixty-four ranks on one model would queue on the cap rather than translate. So rank r runs model $r \bmod 8$ of a pool of eight, at every scale, which multiplies the aggregate limit by eight and makes the executors heterogeneous in a way the protocol never asks about. A segment's model is a function of its rank alone, the mix is the same across the series, and the trace records the model behind every task, so per-model latency and cost are a by-product of the run rather than an extra experiment.

Across machines. The single-node runs use the SQLite device and sixteen to sixty-four processes on a four-core sandbox; the processes are network-bound and the core count does not matter. The multi-node runs use `gitd`, the git device behind a per-node daemon (§7). E5 measured the git device at one push per mutation: thirty-two ranks on thirty-two machines made 1131 pushes and suffered 13415 rejections in forty-two minutes for four collectives. The daemon owns the working tree on each machine and group-commits every mutation its ranks issue within a window—the intra-node aggregation every production MPI performs before it touches the interconnect—so the pushes on the wire are bounded by the machine's rate of bursts, not its ranks' rate of operations. The guarantee each rank sees is unchanged. With stub executors, eight ranks over two machines and a hosted remote made 1371 operations in 441 pushes; the median collective's slowest participant waited a minute, against a second on one machine. That minute is the price of sharing nothing but a git remote, and the question the multi-node runs answer is whether it matters against an executor whose one step costs minutes.

Method. Every run cuts the same book by the same rule, uses the same prompts, the same research cap (48 terms) and budget (3 per rank), the same model pool, and the same policies (quorum 1, barrier policy *proceed*, one restart per rank). What changes with p is the segment size and the number of segments: strong scaling. The launch plan names every rank requested before any runs; each node's launch record carries the machine's kernel boot id; each rank's report carries its epoch and the model it ran on; and the trace carries everything else. The source is in copyright and never leaves the untracked working directory; the repository carries the evidence, the population's glossary, its research findings with their sources, and its amendment ledger.

Results. The population finishes, and the protocol never wedges. Every run in Table 5 closed all of its collectives and assembled a book. At $p = 16$ the sixteen processes rendered 98.5% of the 1232 paragraphs in 51.4 minutes for $7.61; at $p = 64$, 96.0% in 37.9 minutes for $11.06. The coverage that is missing is not the protocol's: it is the segments of ranks whose model degenerated (below), which the runtime dropped from their collectives after the restart budget was spent so that the rest of the population could close.

More ranks bought little wall time, and the trace says why. Wall time fell from 51.4 to 39.9 to 37.9 minutes as p quadrupled, while executor work stayed near 4.2–6.6 rank-hours and blocked time grew from 9.3 to 32.7 rank-hours: the coordination share rose from 68.0% to 81.0%, and parallel efficiency fell from 30.6% to 16.2%. This is not protocol overhead. On one machine the median collective released its slowest participant within a second. It is Amdahl with a heterogeneous denominator: the run ends when its last rank ends, and the last rank is decided by which model it drew. Across the three runs the median translation chunk took 28 s on the fastest model in the pool and 119 s on the slowest, a 4.3× spread (Figure 1); the longest single wait in every run was a seam exchange or the final spend reduction blocked on one rank whose model was slow, or had failed and been restarted. With a fixed book, adding ranks shortens every rank's segment but does not shorten the slowest model's proportion of it.

What the collectives did. The census lifted 22, 53 and 104 conflicts at the three scales—renderings on which ranks reading different parts of the book disagreed—and every one was arbitrated exactly once by a model, in parallel batches, before the research agenda was built from them. The 48 researched terms cite 121 sources at $p = 16$ and were each researched by exactly one rank at every scale: the compare-and-swap claim on the research window is what turned 64 potential investigations of the Singer House into one. The glossary that bound the translation grew from 380 to 1404 terms with p, because more ranks survey more finely; the amendment ledger, written under the per-chapter lock, recorded 360, 276 and 207 terms translators settled themselves, with 2, 1 and 4 clashes—two ranks settling the same term differently in the same chapter— which is exactly the number the glossary phase had failed to prevent, and which the ledger exists to make visible.

Failures were contained, and they were instructive. 3, 2 and 20 ranks were restarted by `ampirun` at the three scales; each successor took a new epoch, re-entered its closed collectives (0 collective re-entries traced as replays across the series), read back its finished tasks, and resumed. Most restarts at $p = 64$ had one cause, a race in the harness that delivered broadcast bodies to a file shared between ranks, found and fixed while the run was in progress; a rank restarted after the fix ran the fixed code, which is a property of process ranks that session ranks do not have. The failures that cost coverage had a different cause: one model in the pool degenerated into thousands of blank lines mid-object on the same paragraphs three times running, and in-conversation repair only reproduced the degeneration. A homogeneous population has no answer to that; a heterogeneous one does, and the executor now falls back to a second model from a fresh conversation when the first exhausts its attempts. The multi-node runs carry that fix; the single-node runs above do not, and the difference is reported rather than hidden.

Rate limits are a shared resource, and the trace measures them. Every capable model on the account was capped at twenty requests a minute, per model. The eight-model pool is what let sixty-four ranks proceed at all; the first attempt on one model lost six ranks to the cap before rate limits were made a wait rather than a failure. The executor now records the time it spent rate-limited, so at $p = 256$ the cost of the cap is a number in the trace rather than an inference from wall time.

Across machines. The multi-node runs did not complete, and what stopped them is the result. Four attempts were made, each one AgentMPI job whose ranks were processes on several cloud machines sharing nothing but a git remote (`runs/e7-rawapi-p128-attempt1`, `p256-attempt1`, `p128-attempt2`, `p256-attempt2`). The first, 128 ranks over four machines, found a defect in the daemon: its readers queued behind the lock its writer held for the whole push contest, and 57 live ranks blocked in one broadcast were convicted for silence. The trace records it exactly—suspicions with `silent_for` between 811 and 1457 s, convictions for `lease_expired`, no retraction, because a rank that cannot complete a read cannot renew its lease—and the fix (readers take no lock when a writer fetched within the read interval) is in the device and the specification (S15.2). The second, 256 ranks over eight machines, found the transport's ceiling: one compare-and-swap ref admits about one commit per round trip for the whole population, and eight daemons contending for it landed one push in ten (node 0: 45 pushes, 442 rejections in 80 minutes), so a rank whose lease renewal was itself a push could not renew it. The third and fourth, 128 over four machines and 256 over four machines of 64, ran without a single spurious conviction; the 128-rank job closed its census, arbitrated its conflicts in parallel and began research, and along the way a node whose machine was recycled rejoined the job and respawned its 32 ranks with fresh epochs while the others waited. Both were then stopped, at 123 and 42 minutes, by

p	nodes	wall (min)	tasks	restarts	coord. share	efficiency	spend ($)	coverage
16	1	51.4	241	3	68.0%	30.6%	7.61	98.5%
32	1	39.9	280	2	73.1%	25.3%	9.40	95.5%
64	1	37.9	366	20	81.0%	16.2%	11.06	96.0%

Table 5: E7 across scales. Coordination share is blocked rank-seconds over all rank-seconds; efficiency is achieved parallelism over p; coverage is the fraction of the 1232 paragraphs rendered into all three languages.

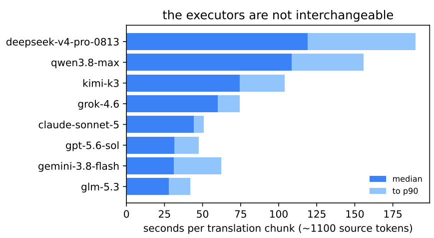

Figure 1: Seconds per translation chunk (about 1100 source tokens into three languages) by model, pooled over the single-node runs: median, and median to 90th percentile. Rank r ran model $r \bmod 8$; the run ends when the slowest of them does.

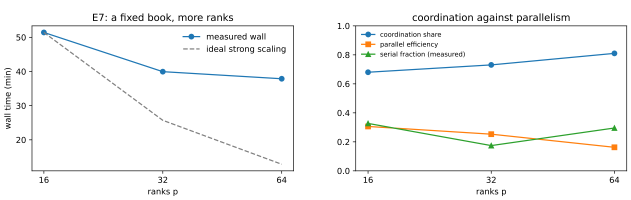

Figure 2: Left: wall time against p for a fixed book, with ideal strong scaling. Right: coordination share, parallel efficiency, and the measured serial fraction.

the account's usage limit freezing every session at once. The measured facts are that a machine hosting 64 rank processes is unremarkable, that four machines are inside the transport's ceiling and eight are outside it, and that the protocol's answer to a lost machine—convict, respawn, replay—worked at 128 ranks with the machine coming back. A completed 256-rank run wants four machines and an account that is not shared with another experiment.

9 WHAT ONLY REAL AGENTS EXPOSED

Several rules in the specification exist because a run failed. They are worth listing separately, because they are the part of this work that could not have been derived from MPI.

(1) **Executors give up far earlier than instructed.** One told to retry up to twenty times gave up after two and stalled its reduction tree. A protocol that depends on an executor's persistence is not a protocol; the binding retries internally and the default is "keep waiting".

(2) **Blocking is not evidence of death.** A rank waiting inside a barrier made no runtime calls, so the detector convicted it for waiting, and every rank that arrived early was declared failed. A blocked rank must renew its own lease.

(3) **Program order is not a collective identifier.** Hence labels.

(4) **Ambient identity without assertion is unsafe.** Shared shell sessions made one rank accumulate nine initialisations, one from nearly every other agent.

(5) **Never truncate a reduction operand.** A result may be summarised because its consumer is a reader; an operand may not, because its consumer is a function. Agents handed JSON clipped mid-string invented recovery hacks, one prefix-matching the content-addressed store by hand.

(6) **Print only commands that exist** — including the bootstrap prompt, which is also a command the binding prints. Ours placed the identity flags before the subcommand, where a reader expects a global option and where our parser rejected them, and roughly thirty executors each lost their first call to `invalid choice`. Our mechanical check walked the strings the runtime emits and not the prompt every executor reads first. And any guard left out of a printed command is a guard that is present only when the agent thinks of it: the emitted `submit` omitted `--expect-rank`, and exactly the executors who noticed had it.

(7) **Give every payload a free path to disk.** Agents that lacked one reached into the object store directly rather than pay to see what was already a file.

(8) **A rank that never starts must still be detectable.**

(9) **A conditional send with an unconditional receive is a deadlock.** The protocol can only bound the damage; the advice is to send unconditionally, possibly empty. The static checker catches it.

(10) **When a protocol omits a mechanism, its users build it worse.** Eight agents given no coordination phase rebuilt an allgather by hand within ninety seconds of one another, then built runtime interface discovery on top at roughly five times the detection code and twice the total lines for the same graded behaviour, with two of the eight introducing defects inside their own detection logic. AgentMPI/1.0 therefore specifies interface declaration *and* verification, because the arms showed they are complementary: declaration alone reproduces the inconsistency of §5, and verification alone pushes the whole cost into every consumer.

10 DISCUSSION AND LIMITATIONS

What we have not shown. We have not shown that AgentMPI produces better answers than an ad-hoc harness. Our one controlled quality comparison is null (§8), and we report it as such. What we have shown is narrower and, we think, still worth having: that MPI's mechanisms have well-defined agent analogues, that several of MPI's answers change in stated and measurable ways, that the resulting interface can be specified precisely enough to be conformance tested, and that real agent populations run against it.

We hot-patched a running job, twice. An executor in the scale run reported an `ImportError` from a module another session was in the middle of fixing. Appendix D of our own specification warns about exactly this, and the warning was inadequate: it said to pin the runtime *version*, and the version string had not changed, because what changed was the code. The runtime now records a content hash of its own source at job creation and logs a diagnostic when a later caller's differs — advisory rather than fatal, because a developer iterating should not be locked out and a two-hour run should not die over a reflowed comment, but recorded, because a run whose runtime changed halfway through should say so in its own journal.

Statistical power. Every agent experiment here is $n = 1$ per arm. Agent runs are expensive and not reproducible, which is also why tracing is unconditional and why we commit raw per-rank artifacts rather than aggregates.

Scale of the agent runs. Our largest *completed* agent run is $p = 32$ over 8 executors. We attempted $p = 100$ and it did not finish: the host's session cap forced several waves of executors, and an operator error later destroyed the run's journal. The pull queue handled the first problem well—a fresh wave could be added mid-run against exactly the ranks still queued, with no harness change and no restart, and 199 tasks were published across 19 executors—and the second was ours, not the protocol's. We report the partial run's identity measurement because it was committed before the journal was lost, and we do not report a hundred-rank result, because we do not have one. Microbenchmarks reach $p = 128$ with scripted ranks; the agent runs do not.

The strongest objection. MPI's SPMD model presupposes a static rank set and interchangeable workers, and agent systems are neither. We think the objection is partly right: the fixed world size is a real constraint, and heterogeneous roles fit awkwardly into a model where rank r's program is the same as rank s's. Our defence is that a rank is a *role* rather than a worker, that communicators already express heterogeneity by partitioning, and that the constraint buys the thing the ad-hoc alternatives lack: a fixed membership against which a collective can be defined at all.

Agreement is not correctness. A protocol that makes agreement cheap makes it cheap to agree on something false. In our translation run a glossary bound a term to a rendering correct nearly everywhere, and a rank correctly obeying it produced a wrong sentence in the one passage with a different sense. A consistency metric scores a uniformly wrong decision as perfect.

11 RELATED WORK

Agent protocols. MCP [37] standardises client–server tool access, and A2A [1] and ACP standardise agent discovery and task handoff. None provides a communicator, a collective, shared state with atomics, failure detection, or flow control; they are complementary rather than competing, and AgentMPI could run over any of them as a transport.

Agent communication languages. KQML [17] and FIPA-ACL [18] define speech acts with mentalistic semantics [30]. The critique that those semantics are unverifiable from outside applies with more force to a language model, and we take MPI's road instead.

Frameworks. AutoGen [45], LangGraph, CrewAI, MetaGPT and their relatives supply coordination *patterns*. Empirical work on how such systems fail [11] independently identifies several of the modes our failure model enumerates. They are the analogue of the pre-MPI libraries: each solves part of the problem, and none composes with another. The distinction we draw is that a framework makes the coordination decisions and a protocol provides the mechanisms with which to make them.

Distributed computing. `Win_fence` is Valiant's BSP superstep boundary [43]. Quorum collectives are stale-synchronous parallelism [26] expressed as a collective. Supervision with a bounded restart intensity is Erlang/OTP's [4, 15]. Recovery by briefing is durable execution [36]. Windows are a blackboard in the sense of HEARSAY-II and Linda [19, 25], with MPI-3's one-sided discipline imposed on it, and work claiming by compare-and-swap is the Contract Net's descendant [40]. We claim the synthesis, not the parts.

12 CONCLUSION

MPI's designers were solving a coordination problem under a particular cost model, and they were unusually explicit about which of their decisions followed from that model. That explicitness is what makes the transplant possible: one can take the mechanism, change the cost model, and see which answers change. Most of them survive. Two selection rules invert, one hierarchy collapses, one non-goal becomes the common case, and one resource with no analogue turns optimisation into feasibility.

The part that did not come from MPI is the part we would most like to be wrong about. A reduction evaluated by agents can be perfectly reproducible and still internally contradictory, because merges are local and invariants are not. We can name the problem, give it a mechanism whose correctness is a two-line semilattice argument, and check that mechanism exhaustively. We cannot make an agent operator correct, and we do not think a protocol should claim to.

REFERENCES

[1] A2A Project. 2026. Agent2Agent (A2A) Protocol Specification, version 1.0.0. Linux Foundation / Agentic AI Foundation. https://a2a-protocol.org/v1.0.0/specification/

[2] Peter Ahrens, James Demmel, and Hong Diep Nguyen. 2020. Algorithms for efficient reproducible floating point summation. *ACM Trans. Math. Software* 46, 3 (July 2020), 22:1–22:49. https://doi.org/10.1145/3389360

[3] Albert Alexandrov, Mihai F. Ionescu, Klaus E. Schauser, and Chris Scheiman. 1995. LogGP: Incorporating long messages into the LogP model — one step closer towards a realistic model for parallel computation. In *Proceedings of the Seventh Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '95)*. ACM, 95–105. https://doi.org/10.1145/215399.215427

[4] Joe Armstrong. 2003. *Making Reliable Distributed Systems in the Presence of Software Errors*. Ph. D. Dissertation. Royal Institute of Technology (KTH), Stockholm, Sweden.

[5] Wesley Bland, Aurelien Bouteiller, Thomas Herault, George Bosilca, and Jack Dongarra. 2013. Post-Failure Recovery of MPI Communication Capability: Design and Rationale. *The International Journal of High Performance Computing Applications* 27, 3 (2013), 244–254. https://doi.org/10.1177/1094342013488238

[6] Wesley Bland, Aurelien Bouteiller, Thomas Herault, Joshua Hursey, George Bosilca, and Jack J. Dongarra. 2012. An Evaluation of User-Level Failure Mitigation Support in MPI. In *Recent Advances in the Message Passing Interface (EuroMPI 2012) (Lecture Notes in Computer Science, Vol. 7490)*. Springer, 193–203. https://doi.org/10.1007/978-3-642-33518-1_24

[7] Guy E. Blelloch. 1990. *Prefix sums and their applications*. Technical Report CMU-CS-90-190. School of Computer Science, Carnegie Mellon University.

[8] George Bosilca, Rémi Delmas, Jack J. Dongarra, and Julien Langou. 2009. Algorithm-Based Fault Tolerance Applied to High Performance Computing. *J. Parallel and Distrib. Comput.* 69, 4 (2009), 410–416. https://doi.org/10.1016/j.jpdc.2008.12.002

[9] Jehoshua Bruck, Ching-Tien Ho, Shlomo Kipnis, Eli Upfal, and Derrick Weathersby. 1997. Efficient algorithms for all-to-all communications in multiport message-passing systems. *IEEE Transactions on Parallel and Distributed Systems* 8, 11 (Nov. 1997), 1143–1156. https://doi.org/10.1109/71.642949

[10] Darius Buntinas, Guillaume Mercier, and William Gropp. 2006. Design and Evaluation of Nemesis, a Scalable, Low-Latency, Message-Passing Communication Subsystem. In *Sixth IEEE International Symposium on Cluster Computing and the Grid (CCGrid '06)*. IEEE, Singapore, 521–530. https://doi.org/10.1109/CCGRID.2006.31

[11] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*. arXiv:2503.13657 https://arxiv.org/abs/2503.13657

[12] Tushar Deepak Chandra and Sam Toueg. 1996. Unreliable Failure Detectors for Reliable Distributed Systems. *J. ACM* 43, 2 (1996), 225–267. https://doi.org/10.1145/226643.226647

[13] David Culler, Richard Karp, David Patterson, Abhijit Sahay, Klaus Erik Schauser, Eunice Santos, Ramesh Subramonian, and Thorsten von Eicken. 1993. LogP: Towards a Realistic Model of Parallel Computation. In *Proceedings of the Fourth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '93)*. ACM, 1–12. https://doi.org/10.1145/155332.155333

[14] Jack J. Dongarra, Rolf Hempel, Anthony J. G. Hey, and David W. Walker. 1993. *A Proposal for a User-Level, Message-Passing Interface in a Distributed Memory Environment*. Technical Report ORNL/TM-12231. Oak Ridge National Laboratory.

[15] Ericsson AB. 2025. Supervisor Behaviour — Erlang/OTP System Documentation. https://www.erlang.org/docs/29/system/sup_princ.html.

[16] Graham E. Fagg and Jack J. Dongarra. 2000. FT-MPI: Fault Tolerant MPI, Supporting Dynamic Applications in a Dynamic World. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface (EuroPVM/MPI 2000) (Lecture Notes in Computer Science, Vol. 1908)*. Springer, 346–353. https://doi.org/10.1007/3-540-45255-9_47

[17] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. 1994. KQML as an Agent Communication Language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM '94)*. ACM, Gaithersburg, Maryland, USA, 456–463. https://doi.org/10.1145/191246.191322

[18] Foundation for Intelligent Physical Agents. 2002. *FIPA ACL Message Structure Specification*. Technical Report SC00061G. Foundation for Intelligent Physical Agents. http://www.fipa.org/specs/fipa00061/SC00061G.html

[19] David Gelernter. 1985. Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems* 7, 1 (Jan. 1985), 80–112. https://doi.org/10.1145/2363.2433

[20] Cary G. Gray and David R. Cheriton. 1989. Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP '89)*. ACM, 202–210. https://doi.org/10.1145/74850.74870

[21] William Gropp and Ewing Lusk. 1994. *An Abstract Device Definition to Support the Implementation of a High-Level Point-to-Point Message-Passing Interface*. Technical Report Preprint MCS-P342-1193. Mathematics and Computer Science Division, Argonne National Laboratory, Argonne, IL, USA.

[22] William Gropp, Ewing Lusk, Nathan Doss, and Anthony Skjellum. 1996. A High-Performance, Portable Implementation of the MPI Message Passing Interface Standard. *Parallel Comput.* 22, 6 (1996), 789–828. https://doi.org/10.1016/0167-8191(96)00024-5

[23] William D. Gropp. 2001. Learning from the Success of MPI. In *High Performance Computing — HiPC 2001, 8th International Conference, Hyderabad, India, December 17–20, 2001 (Lecture Notes in Computer Science, Vol. 2228)*, Burkhard Monien, Viktor K. Prasanna, and Sriram Vajapeyam (Eds.). Springer, Berlin, Heidelberg, 81–92. https://doi.org/10.1007/3-540-45307-5_8

[24] William D. Gropp and Barry Smith. 1993. *Chameleon Parallel Programming Tools Users Manual*. Technical Report ANL-93/23. Argonne National Laboratory.

[25] Barbara Hayes-Roth. 1985. A Blackboard Architecture for Control. *Artificial Intelligence* 26, 3 (1985), 251–321. https://doi.org/10.1016/0004-3702(85)90063-3

[26] Qirong Ho, James Cipar, Henggang Cui, Jin Kyu Kim, Seunghak Lee, Phillip B. Gibbons, Garth A. Gibson, Gregory R. Ganger, and Eric P. Xing. 2013. More Effective Distributed ML via a Stale Synchronous Parallel Parameter Server. In *Advances in Neural Information Processing Systems 26 (NIPS 2013)*. 1223–1231.

[27] Torsten Hoefler and Marc Snir. 2011. Writing Parallel Libraries with MPI — Common Practice, Issues, and Extensions. In *Recent Advances in the Message Passing Interface, 18th European MPI Users' Group Meeting, EuroMPI 2011, Santorini, Greece, September 18–21, 2011 (Lecture Notes in Computer Science, Vol. 6960)*, Yiannis Cotronis, Anthony Danalis, Dimitrios S. Nikolopoulos, and Jack Dongarra (Eds.). Springer, Berlin, Heidelberg, 345–355. https://doi.org/10.1007/978-3-642-24449-0_45

[28] Kuang-Hua Huang and Jacob A. Abraham. 1984. Algorithm-Based Fault Tolerance for Matrix Operations. *IEEE Trans. Comput.* C-33, 6 (1984), 518–528. https://doi.org/10.1109/TC.1984.1676475

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*. ACM, 611–626. https://doi.org/10.1145/3600006.3613165

[30] Yannis Labrou and Tim Finin. 1994. A Semantics Approach for KQML — A General Purpose Communication Language for Software Agents. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM '94)*. ACM, Gaithersburg, Maryland, USA, 447–455. https://doi.org/10.1145/191246.191320

[31] Richard E. Ladner and Michael J. Fischer. 1980. Parallel prefix computation. *J. ACM* 27, 4 (Oct. 1980), 831–838. https://doi.org/10.1145/322217.322232

[32] Ignacio Laguna, Ryan Marshall, Kathryn Mohror, Martin Ruefenacht, Anthony Skjellum, and Nawrin Sultana. 2019. A Large-Scale Study of MPI Usage in Open-Source HPC Applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '19)*. ACM, New York, NY, Article 31, 14 pages. https://doi.org/10.1145/3295500.3356176

[33] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost

in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. https://doi.org/10.1162/tacl_a_00638

[34] Message Passing Interface Forum. 1994. *MPI: A Message-Passing Interface Standard*. Technical Report CS-94-230. University of Tennessee, Knoxville.

[35] Message Passing Interface Forum. 2023. *MPI: A Message-Passing Interface Standard, Version 4.1*. Technical Report. University of Tennessee, Knoxville. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf

[36] Microsoft. 2025. Durable Functions Overview. https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview.

[37] Model Context Protocol Contributors. 2026. Model Context Protocol Specification, revision 2026-07-28. Agentic AI Foundation / Linux Foundation. https://modelcontextprotocol.io/specification/2026-07-28/basic/index

[38] Rolf Rabenseifner. 2004. Optimization of collective reduction operations. In *Computational Science — ICCS 2004 (Lecture Notes in Computer Science, Vol. 3036)*. Springer, 1–9. https://doi.org/10.1007/978-3-540-24685-5_1

[39] Rolf Rabenseifner and Jesper Larsson Träff. 2004. More efficient reduction algorithms for non-power-of-two number of processors in message-passing parallel systems. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface (EuroPVM/MPI 2004) (Lecture Notes in Computer Science, Vol. 3241)*. Springer, 36–46. https://doi.org/10.1007/978-3-540-30218-6_13

[40] Reid G. Smith. 1980. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Trans. Comput.* C-29, 12 (Dec. 1980), 1104–1113. https://doi.org/10.1109/TC.1980.1675516

[41] Marc Snir, Steve Otto, Steven Huss-Lederman, David Walker, and Jack Dongarra. 1996. *MPI: The Complete Reference*. MIT Press, Cambridge, MA.

[42] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. 2005. Optimization of collective communication operations in MPICH. *The International Journal of High Performance Computing Applications* 19, 1 (Feb. 2005), 49–66. https://doi.org/10.1177/1094342005051521

[43] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Commun. ACM* 33, 8 (Aug. 1990), 103–111. https://doi.org/10.1145/79173.79181

[44] David W. Walker. 1992. *Standards for Message-Passing in a Distributed Memory Environment*. Technical Report ORNL/TM-12147. Oak Ridge National Laboratory, Oak Ridge, TN.

[45] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] https://arxiv.org/abs/2308.08155